

COMPUTER NETWORKS

Tanenbaum · Feamster · Wetherall — 6th Edition (2021)

Chapter 3

THE DATA LINK LAYER

Complete textual study notes — every section, concept, algorithm, formula, and protocol detail

Covers §3.1 – §3.6 (book pp. 201–266)

Contents

Contents	2
Chapter 3 at a Glance	4
3.1 Data Link Layer Design Issues	4
3.1.1 Services Provided to the Network Layer	4
3.1.2 Framing	5
3.1.3 Error Control	6
3.1.4 Flow Control	7
3.2 Error Detection and Correction	7
3.2.1 Error-Correcting Codes	8
Hamming distance	8
The Hamming code and the single-error bound	9
Binary convolutional codes	9
Reed-Solomon codes	10
LDPC (Low-Density Parity Check) codes	10
3.2.2 Error-Detecting Codes	11
Parity	11
Checksums	11
CRC (Cyclic Redundancy Check) / polynomial code	12
3.3 Elementary Data Link Protocols	13
3.3.1 Initial Simplifying Assumptions	13
3.3.2 Basic Transmission and Receipt	14
3.3.3 Simplex Link-Layer Protocols	15
Protocol 1 — Utopia: no flow control or error correction	15
Protocol 2 — Stop-and-Wait: adding flow control	15
Protocol 3 — PAR / ARQ: adding error control with sequence numbers	16
3.4 Improving Efficiency	17
3.4.1 Goal: Bidirectional Transmission, Multiple Frames in Flight	17
Bidirectional transmission: piggybacking	17
Sliding windows	17
3.4.2 Examples of Full-Duplex, Sliding Window Protocols	18
Protocol 4 — One-bit sliding window	18
Motivation for pipelining: the bandwidth-delay product	18
Go-Back-N (Protocol 5)	19
Selective Repeat (Protocol 6)	19
3.5 Data Link Protocols in Practice	20
3.5.1 Packet over SONET	20
3.5.2 ADSL (Asymmetric Digital Subscriber Loop)	22
3.5.3 DOCSIS (Data Over Cable Service Interface Specification)	22

3.6 Summary..... 23

Chapter 3 at a Glance

What the data link layer is for, and the road map of the chapter.

The **data link layer** is the second layer in the reference model. Its job is to achieve **reliable, efficient communication of whole units of information called frames** (rather than individual bits, as in the physical layer) between two **adjacent** machines — machines connected by a communication channel that acts conceptually like a wire (coaxial cable, telephone line, or wireless channel). The essential "wire-like" property is that **bits are delivered in exactly the same order in which they are sent**.

This seems trivial ("A puts bits on the wire, B takes them off"), but real channels (1) make occasional errors, (2) have a finite data rate, and (3) have a nonzero propagation delay between sending and receiving a bit. These limitations shape the efficiency of data transfer, and the protocols must take all of them into account.

How this chapter is organised

First the key design issues of the link layer (§3.1); then the nature of errors and how they are detected and corrected (§3.2); then a series of increasingly complex example protocols, each solving more of the problems (§3.3–§3.4); finally real-world link protocols — PPP over SONET, ADSL, and DOCSIS (§3.5), and a summary (§3.6).

3.1 Data Link Layer Design Issues

The data link layer uses the services of the physical layer below it to send and receive bits over (possibly unreliable) communication channels that may lose data. It has four principal functions:

1. **Providing a well-defined service interface to the network layer** (§3.1.1).
2. **Framing** sequences of bytes as self-contained segments (§3.1.2).
3. **Detecting and correcting transmission errors** (§3.1.3).
4. **Regulating the flow of data** so slow receivers are not swamped by fast senders — flow control (§3.1.4).

To do this, the link layer takes the packets it gets from the network layer and **encapsulates** them into frames. Each frame contains a **frame header**, a **payload field** holding the packet, and a **frame trailer** (Fig. 3-1). Frame management is the heart of what the data link layer does.

Why study these principles here

Error control and flow control are also found in the transport and other layers — reliability is an overall goal achieved when all layers work together. In many networks these functions live mostly in the upper layers, with the link layer doing the minimal job that is "good enough." But the principles are the same everywhere, and they show up in their simplest, purest form at the link layer, making it a good place to examine them.

3.1.1 Services Provided to the Network Layer

The link layer's purpose is to transfer data from the network layer on the source machine to the network layer on the destination machine.

On the source machine a process in the network layer hands packets to the data link layer; the link layer's job is to transmit them to the destination so they can be handed to the network layer there. It is easiest to think of two data-link-layer processes communicating using a data link protocol — this is **virtual communication**. The **actual communication** path goes down the sender's stack, across the wire, and up the receiver's stack, but we use the virtual model throughout the chapter.

The link layer can offer three reasonable kinds of service:

Service	Connection?	Acknowledged?	Key properties & example
Unacknowledged connectionless	No	No	Source sends independent frames; destination does not acknowledge. No connection set up or released. A frame lost to noise is not recovered at the link layer. Good when the error rate is very low (recovery left to higher layers) and for real-time traffic (voice/video) where late data are worse than bad data. Example: Ethernet .
Acknowledged connectionless	No	Yes (per frame)	No logical connection, but every frame is individually acknowledged, so the sender knows whether each arrived. If an ack does not come within a time interval, the frame is resent. Useful over unreliable channels. Example: 802.11 (WiFi) .
Acknowledged connection-oriented	Yes	Yes	A connection is established before data transfer; each frame is numbered. The link layer guarantees every frame is received exactly once and in the right order — i.e. a reliable bit stream. Appropriate over long, unreliable links (satellite, long-distance telephone circuits).

Acknowledgements are an optimization, never a requirement

The network layer could always send a packet and wait for its peer to acknowledge it, retransmitting the whole message on timeout. The trouble is efficiency: links often impose a strict maximum frame length and have known propagation delays that the network layer does not know. A large packet broken into ~10 frames, of which 2 are lost on average, would take a very long time to get through. Acknowledging and retransmitting individual frames corrects errors more directly and quickly. On reliable channels (fiber) a heavyweight link protocol may be unnecessary; on inherently unreliable wireless channels the overhead is usually worth the cost.

Connection-oriented transfers go through three phases: (1) the connection is **established** — both sides initialize variables and counters that track which frames have and have not been received; (2) one or more **frames are transmitted**; (3) the connection is **released**, freeing the variables, buffers, and other resources.

3.1.2 Framing

Turning the physical layer's raw, error-prone bit stream into discrete frames the receiver can delimit.

The physical layer accepts a raw bit stream and tries to deliver it, adding redundancy on noisy channels to reduce the bit error rate — but the bit stream the link layer receives is **not guaranteed error-free**: bits may change value, and the number of bits received may be less than, equal to, or more than the number transmitted. The link layer must detect and, if necessary, correct errors.

The usual approach: break the bit stream into discrete frames, compute a short token called a **checksum** for each frame, and include it in the frame. The receiver recomputes the checksum; if it differs from the one in the frame, an error has occurred and the frame is dealt with (e.g. discarded, possibly with an error report sent back). A good framing design must make it easy for the receiver to find the start of a new frame while using little channel bandwidth. Four methods:

Framing method	How it works	Weakness / notes
1. Byte count	A header field specifies the number of bytes in the frame; the receiver counts that many bytes to find the end.	A single bit-flip can garble the count, so the receiver loses synchronization and cannot find the next frame's start —

Framing method	How it works	Weakness / notes
		and a retransmission request does not help because it does not know how many bytes to skip. Rarely used by itself.
2. Flag bytes with byte stuffing	Each frame starts and ends with a special FLAG byte (two consecutive flags = end of one, start of next). An accidental FLAG inside the data is preceded by an ESC (escape) byte; an ESC in the data is itself preceded by an ESC.	Byte-oriented (tied to 8-bit bytes). Receiver removes stuffed ESCs before passing data up; sequence after destuffing equals the original. Used (in a slightly simplified form) by PPP .
3. Flag bits with bit stuffing	Framing at the bit level, so frames may have any number of bits. Each frame is delimited by the bit pattern 01111110 (0x7E). The sender stuffs a 0 after any five consecutive 1s in the data; the receiver deletes a 0 that follows five 1s.	Developed for HDLC . Also guarantees a minimum density of transitions for physical-layer sync (USB uses bit stuffing for this). Transparent to the network layer: data 01111110 goes on the line as 011111010.
4. Physical-layer coding violations	Use signals that cannot occur in regular data as frame delimiters. E.g. the 4B/5B line code maps 4 data bits to 5 signal bits, so 16 of 32 signals are unused and can be reserved to mark frame start/end.	“Coding violations” (invalid characters). Easy to find frame start/end and no data stuffing is needed.

Side effect of stuffing: a frame’s length now depends on its contents. If the data contain no flag bytes, 100 bytes travel in a ~100-byte frame; if the data are solely flag bytes, each is escaped and the frame becomes ~200 bytes. With bit stuffing the increase is roughly **12.5%** (about 1 bit added per byte).

Combining methods for safety: Many protocols combine techniques. A common pattern in Ethernet and 802.11 is to begin a frame with a **preamble** — a well-defined pattern (72 bits is typical for 802.11) that lets the receiver prepare for an incoming packet — followed by a length (count) field in the header that locates the end of the frame.

3.1.3 Error Control

Making sure every frame is eventually delivered to the destination network layer, in order, exactly once.

Assume the receiver can tell whether a frame it receives is correct or faulty (the codes that do this are in §3.2). For unacknowledged connectionless service the sender can just keep outputting frames; for reliable, connection-oriented service it cannot. The usual way to ensure reliable delivery is to give the sender **feedback**: the receiver returns special control frames carrying **positive** or **negative acknowledgements**. A positive ack means the frame arrived safely; a negative ack means something went wrong and the frame must be sent again.

A complication is that hardware trouble (e.g. a noise burst) can make a frame **vanish completely** — the receiver has no reason to react — or the acknowledgement itself can be lost. A protocol that simply transmits a frame and waits for an acknowledgement would then **hang forever**. The remedy has three parts:

- **Timers.** When the sender transmits a frame it starts a timer set to expire after enough time for the frame to reach the destination, be processed, and have the ack propagate back. Normally the ack returns first and the timer is canceled; if the frame or its ack is lost, the timer goes off and the sender retransmits.

- **Sequence numbers.** Because retransmissions can cause the receiver to accept the same frame two or more times (and pass a duplicate up to the network layer), outgoing frames carry sequence numbers so the receiver can distinguish a retransmission from an original.
- **Exactly-once delivery.** Managing the timers and sequence numbers so every frame is ultimately passed to the destination network layer exactly once — no more, no less — is a central duty of the link layer (and of higher layers).

3.1.4 Flow Control

Preventing a fast sender from systematically overrunning a slow receiver.

This happens when a fast, powerful machine sends to a slow one — e.g. a smartphone requests a web page and a far more powerful server “turns on the fire hose” until the phone is swamped. Even with error-free transmission the receiver may be unable to handle frames as fast as they arrive and will lose some. Two approaches are used:

- **Feedback-based flow control** — the receiver sends information back to the sender giving it permission to send more data, or at least telling the sender how the receiver is doing.
- **Rate-based flow control** — the protocol has a built-in mechanism that limits the rate at which the sender may transmit, without any feedback from the receiver.

This chapter studies **feedback-based** schemes; rate-based schemes appear only in the transport layer (Chap. 5). Feedback-based control is seen at both the link layer and higher layers — the latter is more common today, where link hardware is built to run fast enough not to cause loss. Hardware link implementations (**NICs** — **Network Interface Cards**) are said to run at “**wire speed**,” handling frames as fast as they arrive; any overruns are then a higher-layer problem. Most feedback schemes share one principle: well-defined rules about when the sender may transmit the next frame, often prohibiting transmission until the receiver grants permission — e.g. “You may send me n frames now, but after that, send no more until I tell you to continue.”

3.2 Error Detection and Correction

Communication channels vary widely. Optical fiber in telecom networks has such tiny error rates that transmission errors are rare; other channels — especially **wireless links and aging local loops** — have error rates orders of magnitude larger, so for them errors are the norm. They cannot be avoided at reasonable expense or performance cost, so the conclusion is blunt: **transmission errors are here to stay**, and we must learn to deal with them.

Designers use **two basic strategies**, both of which add **redundant information** to the data sent. One includes enough redundancy for the receiver to **deduce what the transmitted data must have been** (error-correcting codes, also called **FEC — Forward Error Correction**). The other includes only enough redundancy for the receiver to **deduce that an error occurred** — but not which error — and then **request a retransmission** (error-detecting codes).

Which strategy, and why

Each occupies a different ecological niche. On highly reliable channels (e.g. fiber) it is cheaper to use an error-detecting code and just retransmit the occasional faulty block. On channels that make many errors (e.g. wireless) it is better to add enough redundancy for the receiver to reconstruct the original — FEC is used on noisy channels precisely because a retransmission would be just as likely to be in error as the first transmission.

A key consideration is the **type of errors likely to occur**. Neither code can handle all possible errors, because the redundant bits that offer protection are **as likely to be received in error as the data bits** — the channel treats them all simply as bits. So to avoid undetected errors, the code must be strong enough for the errors expected. Two error models matter in practice:

- **Isolated single-bit errors** — caused by extreme values of thermal noise that briefly and occasionally overwhelm the signal.
- **Burst errors** — errors that come in bursts rather than singly, arising from physical processes such as a deep fade on a wireless channel or transient electrical interference on a wire. Advantage: since data are sent in blocks, bursts concentrate the damage — with a 1000-bit block and an error rate of 0.001 per bit, independent errors would corrupt most blocks, but bursts of 100 affect only about one block in 100. Disadvantage: when bursts occur they are much harder to correct than isolated errors.

A further case is the **erasure channel**: the location of an error is known because the physical layer received an analog value far from a clean 0 or 1 and declared the bit **lost**. Erasures are easier to correct than bit-flips — even though the bit value is gone, we at least know **which** bit is wrong. We often do not have this benefit, however.

Two points to keep in mind

First, these codes appear here because the link layer is the first place we must reliably transmit groups of bits — but they are used throughout: error-correcting codes also appear in the physical layer (noisy channels) and higher layers (real-time media, content distribution), and error-detecting codes are common in the link, network, and transport layers. Second, error codes are applied mathematics — unless you are adept at Galois fields or the properties of sparse matrices, get codes with good properties from a reliable source rather than inventing your own. Many standards do exactly that, with the same codes recurring again and again.

3.2.1 Error-Correcting Codes

Four codes: Hamming, binary convolutional, Reed-Solomon, and LDPC.

All add redundancy. A frame has **m data (message) bits** and **r redundant (check) bits**. Three defining properties, in increasing specificity:

- **Block code** — the r check bits are computed **solely as a function of the m data bits** they are associated with (as though the m bits were looked up in a large table to find their r check bits).
- **Systematic code** — the m data bits are **sent directly**, along with the check bits, rather than being encoded themselves before transmission.
- **Linear code** — the r check bits are a **linear function** of the m data bits; XOR / modulo-2 addition is the popular choice, so encoding is just matrix multiplications or simple logic circuits.

The codes in this section are **linear, systematic block codes** unless noted. Let the total block length be $n = m + r$; this is an **(n, m) code**, and the n -bit unit of data + check bits is a **codeword**. The **code rate** (or rate) is the non-redundant fraction, m / n — about 1/2 for a noisy channel (half the received bits are redundant) up to nearly 1 for a high-quality channel.

Hamming distance

Given any two codewords, the number of bit positions in which they differ is the **Hamming distance** (Hamming, 1950). Compute it by XOR-ing the two codewords and counting the 1s — e.g. **10001001** XOR **10110001** = **00111000**, so they differ in **3** positions. Its significance: if two codewords are distance **d** apart, it takes **d single-**

bit errors to convert one into the other. The smallest Hamming distance over all pairs of legal codewords is the **Hamming distance of the complete code**.

Normally all 2^m data messages are legal, but because of how check bits are computed, only $2^m / 2^n = 1/2^r$ of the 2^n possible codewords are legal. It is this **sparseness** — legal codewords embedded thinly in codeword space — that lets the receiver detect and correct errors. The detecting/correcting power depends on the code's Hamming distance:

- **To reliably detect d errors**, you need a distance $d + 1$ code — then no set of d single-bit errors can turn one valid codeword into another valid one, so any illegal codeword flags an error.
- **To correct d errors**, you need a distance $2d + 1$ code — the legal codewords are then so far apart that even after d changes the original is still the closest codeword, and can be uniquely identified (on the assumption that fewer errors are more likely).

Example. A code with just four valid codewords — `0000000000`, `0000011111`, `1111100000`, `1111111111` — has distance 5, so it can **correct double errors or detect quadruple errors**. If `0000000111` arrives and only single/double errors are expected, the receiver knows the original was `0000011111`. But a triple error turning `0000000000` into `0000000111` would be corrected wrongly. You cannot do both at once — correcting double errors and detecting quadruple errors would require interpreting one received codeword in two different ways.

The Hamming code and the single-error bound

To design a code that corrects all single errors: each of the 2^m legal messages has n illegal codewords at distance 1 (invert each of the n bits), so each legal message needs $n + 1$ bit patterns dedicated to it. Since there are only 2^n patterns in total, $(n + 1) \cdot 2^m \leq 2^n$, and with $n = m + r$ this becomes the bound

$$(m + r + 1) \leq 2^r \quad (\text{Eq. 3-1})$$

which puts a **lower limit on the number of check bits** needed to correct single errors. **Hamming codes** achieve this limit exactly. Number the codeword bits consecutively from **bit 1 at the left**. Bits that are **powers of 2** (1, 2, 4, 8, 16, ...) are **check bits**; the rest (3, 5, 6, 7, 9, ...) hold the m data bits. Each check bit forces the **modulo-2 sum (parity)** of some collection of bits — including itself — to be even (or odd), and a bit may take part in several checks. To find which check bits a data bit in position k contributes to, write k as a **sum of powers of 2**: e.g. $11 = 1 + 2 + 8$ and $29 = 1 + 4 + 8 + 16$, so bit 11 is checked by bits 1, 2, and 8.

A worked **(11, 7) Hamming code** (7 data + 4 check bits, even parity) encoding the ASCII letter **A** gives the sent codeword `00101000001` (check bits in positions 1, 2, 4, 8). This construction has **Hamming distance 3**, so it corrects single errors (or detects double errors).

Decoding with the syndrome

When a codeword arrives, the receiver redoes every check-bit computation, including the received check bits — these are the check results. For even-parity sums each result should be 0; if all are 0 the codeword is accepted. If not, the set of check results is the error syndrome, which pinpoints the bad bit. In the book's example the check results for $k = 8, 4, 2, 1$ are 0, 1, 0, 1 — a syndrome of `0101` = $4 + 1 = 5$ — so bit 5 is in error. Flip it (it may be a data or a check bit), discard the check bits, and the correct message (ASCII A) remains. Hamming codes are used in error-correcting memory, but most networks use stronger codes.

Binary convolutional codes

The only **non-block** code here. An encoder processes a **stream** of input bits and emits a **stream** of output bits — there is no natural message size or encoding boundary. The output depends on the current **and previous**

input bits, so the encoder has **memory**; the number of previous bits the output depends on is the **constraint length**. A convolutional code is specified by its **rate** and **constraint length**, and such codes are widely deployed — GSM, satellite communications, and 802.11.

The example is the **NASA convolutional code**, **rate $r = 1/2$** , **constraint length $k = 7$** , first used on the Voyager missions (1977) and later reused in 802.11. Each input bit produces **two output bits** that are XOR sums of the input and the internal state — a **binary, linear** convolutional code; it is **not systematic**, since neither output bit is simply the input bit. The internal state lives in **six memory registers**, shifted right on each input. Starting from all zeros and inputting **111**, the state becomes **100000, 110000, 111000**, with outputs **11, 10, 01**; it takes **seven shifts** to flush an input completely — hence **$k = 7$** .

Decoding finds the input sequence most likely to have produced the observed output (errors included). For small k this uses the **Viterbi algorithm** (Forney, 1973), which walks the observed sequence keeping, for each step and each possible internal state, the input sequence that would reproduce the observed output with the **fewest errors**; the one needing the fewest errors overall is the most likely message. Convolutional codes also support **soft-decision decoding**: instead of forcing each received level to 0/1 immediately (e.g. +0.9 V = “very likely 1”, -0.1 V = “maybe 0”), the decoder keeps the uncertainty and corrects the sequence as a whole. Deciding each bit before correction is **hard-decision decoding**.

Reed-Solomon codes

Linear block codes, often systematic, that operate on **m-bit symbols** rather than individual bits. They rely on the fact that every polynomial of degree n is **uniquely determined by $n + 1$ points** — a line $ax + b$ is fixed by two points, and extra points on it are redundant. Send two data points plus two check points on the same line; if one point is received in error, fit a line to the received points (three lie on it, the erroneous one does not) and the error is corrected. Reed-Solomon codes are defined as polynomials over **finite fields** but work this way.

For m -bit symbols a codeword is **$2^m - 1$ symbols** long; choosing **$m = 8$** makes symbols bytes and a codeword **255 bytes**. The widely used **(255, 233)** code adds redundant check symbols to 233 data symbols, and decoding uses the **Berlekamp-Massey algorithm** (Massey, 1969). Reed-Solomon codes are strong, especially for **burst errors**, and are used in DSL, data over cable, satellite, and ubiquitously on CDs, DVDs, and Blu-ray. Because they work on symbols, a single-bit error and an m -bit burst are **both just one symbol error**: with **$2t$ redundant symbols** the code corrects up to **t symbol errors** — the (255, 233) family with **32 redundant symbols corrects up to 16 symbol errors**, i.e. a consecutive burst of up to **128 bits**; with **erasures**, up to **$2t$ errors** can be corrected. They are often nested **inside a convolutional code** — the convolutional code handles isolated bit errors but fails in bursts, and the inner Reed-Solomon code mops up those bursts, giving good protection against both.

LDPC (Low-Density Parity Check) codes

Linear block codes invented by **Robert Gallager** in his 1962 doctoral thesis, then forgotten and **reinvented in 1995** once computers were powerful enough. Each output bit is formed from only a **fraction** of the input bits, giving a parity-check matrix with a **low density of 1s** — hence the name. Decoding uses an **iterative approximation** that successively improves the best fit of the received data to a legal codeword. LDPC codes are practical for **large block sizes** and have **excellent** error correction that outperforms the other codes here, so they are being adopted rapidly — digital video broadcasting, 10-Gbps Ethernet, power-line networks, and the latest 802.11.

Code	Kind	Core idea / decoding	Representative uses
Hamming	Linear systematic block	Powers-of-2 check bits; syndrome pinpoints the bad bit. Distance 3 → corrects 1 error. Meets the $(m+r+1) \leq 2^r$ bound.	Error-correcting memory
Convolutional (NASA $r=1/2$, $k=7$)	Non-block, streaming, with memory	XOR of current + previous bits; Viterbi decoding; soft- vs hard-decision.	GSM, satellite, 802.11, Voyager
Reed-Solomon (255, 233)	Linear block, symbol-based	Polynomial through points over a finite field; Berlekamp-Massey; $2t$ check symbols → t symbol errors.	DSL, cable, satellite, CD/DVD/Blu-ray
LDPC	Linear block, sparse matrix	Each output bit from a few input bits; iterative best-fit decoding; superb at large block sizes.	DVB, 10G Ethernet, power-line, latest 802.11

3.2.2 Error-Detecting Codes

On fiber or high-quality copper the error rate is low, so detection + retransmission is usually more efficient than correction.

Three codes, all **linear, systematic block codes**: **parity**, **checksums**, and **cyclic redundancy checks (CRCs)**.

Parity

Append a **single parity bit** chosen so the number of 1s in the codeword is even (or odd) — equivalently, the **XOR of the data bits**. E.g. **1011010** → **10110100** (even) or **10110101** (odd). A single parity bit gives **distance 2**, so it detects any single-bit error.

Why detection can be far cheaper than correction

Take isolated errors at 10^{-6} per bit (a fair rate for a long wired cable; typical LAN links reach 10^{-10}) and 1000-bit blocks. Error correction via a Hamming code needs 10 check bits per block (Eq. 3-1), so a megabit costs 10,000 check bits. Mere detection needs one parity bit per block; about 1 block in 1000 is bad and must be retransmitted (an extra 1001 bits). Total overhead for detect-and-retransmit ≈ 2001 bits per megabit, versus 10,000 bits for the Hamming code.

The weakness: a single parity bit reliably detects only a **single-bit** error — against a long burst the chance of detection is only **0.5**. Treating the block as a **rectangular matrix n wide by k high** and sending **one parity bit per row** detects up to k -bit errors, as long as there is at most one error per row. Better still is **interleaving**:

Compute the parity bits **over the data in a different order** than they are transmitted. Here one parity bit is computed **per column**; the data are sent as k rows (top to bottom, left to right), then the n column-parity bits last. This is a **general technique to turn a code that handles isolated errors into one that handles burst errors**: a burst of length n spreads its errors **across different columns**, so at most one bit per column is affected and the column parities catch it. The method uses **n parity bits over kn data bits** to detect any single burst of length $\leq n$. (A burst does not mean every bit is wrong — only that at least the first and last are.) A burst of length $n + 1$ slips through only if the first and last bits are inverted and all others are correct; for long or multiple bursts the probability a bad block is wrongly accepted is **2^{1-n}** .

Checksums

“Checksum” means any group of check bits associated with a message (a parity group is one example), but stronger checksums use a **running sum of the data bits**, placed at the end as the **complement** of the sum — so summing the entire received codeword (data + checksum) yields **zero** if no error is detected.

The **16-bit Internet checksum** (used on all IP packets; Braden et al., 1988) sums the message as **16-bit words**. Because it works on words, not bits, errors that leave **parity unchanged** can still change the sum and be caught — e.g. flipping the lowest bit of **two different words** from 0 to 1 passes a parity check but adds two 1s to the checksum. It is computed in **one's complement arithmetic** (not modulo 2^{16}): a negative number is the bitwise complement of its positive counterpart. On a two's-complement machine (where a negative is the one's complement plus one), the one's-complement sum equals the sum modulo 2^{16} **with any high-order overflow added back into the low-order bits**, giving more uniform coverage — otherwise two high-order bits could overflow and be lost without changing the sum. A bonus: one's complement has **two representations of zero** (all 0s, all 1s), so one value (e.g. all 0s) can mean “no checksum” without an extra field.

For decades, checksum analyses assumed **random** frame bits; inspection of real data by Partridge et al. (1995) showed that assumption is wrong, so undetected errors are in some cases far more common than thought. The Internet checksum is simple and efficient but weak precisely because it is a simple sum: it does **not** detect the addition or deletion of zero data, nor the swapping of parts of the message, and it gives weak protection against **message splices** (parts of two packets joined) — just the kinds of errors buggy hardware produces. A better choice is **Fletcher's checksum** (Fletcher, 1982), which adds a **positional component** — the product of each data item and its position — to the running sum, strengthening detection of changes in the position of data.

CRC (Cyclic Redundancy Check) / polynomial code

The stronger code in widespread link-layer use. Treat a **k-bit frame as the coefficients of a polynomial** with k terms, from x^{k-1} down to x^0 ; the high-order (leftmost) bit is the coefficient of x^{k-1} . E.g. **110001** represents $x^5 + x^4 + 1$. **Polynomial arithmetic is done modulo 2** (algebraic field theory): no carries or borrows, and both **addition and subtraction are identical to XOR**. Long division proceeds as in binary, with the subtraction done modulo 2; a divisor “goes into” a dividend whenever the dividend has as many bits as the divisor.

Sender and receiver agree in advance on a **generator polynomial $G(x)$** , whose **high- and low-order bits must both be 1**, and the frame must be longer than $G(x)$. The idea is to append a CRC so the transmitted polynomial is **exactly divisible by $G(x)$** ; the receiver divides by $G(x)$ and a **nonzero remainder** signals an error. The algorithm:

1. Let **r = degree of $G(x)$** . Append **r zero bits** to the low-order end of the frame, giving $m + r$ bits — the polynomial **$x^r \cdot M(x)$** .
2. **Divide** the bit string for $G(x)$ into the bit string for $x^r \cdot M(x)$ using **modulo-2 division**.
3. **Subtract** the remainder (always r bits or fewer) from $x^r \cdot M(x)$ using modulo-2 subtraction. The result is the **checksummed frame $T(x)$** to transmit.

Worked CRC (Fig. 3-9)

Frame $M = 1101011111$, generator $G(x) = x^4 + x + 1 =$ bit string 10011, so $r = 4$. Append four zeros $\rightarrow 11010111110000$, divide by 10011 modulo 2, and the remainder is 0010. Subtracting it gives the transmitted frame 11010111110010. The receiver divides this by 10011 and gets remainder 0000 — no error. $T(x)$ is divisible by $G(x)$ because diminishing any dividend by its remainder leaves something divisible by the divisor.

Error analysis. If $T(x)$ is corrupted to $T(x) + E(x)$, each 1 in $E(x)$ is an inverted bit (k ones $\Rightarrow k$ single-bit errors; a single burst is an initial 1, a mix, and a final 1, rest 0). The receiver computes $[T(x) + E(x)] / G(x) = E(x) / G(x)$, since $T(x)/G(x) = 0$ — so only errors whose polynomial has **$G(x)$ as a factor** slip through; all others are caught:

- **Single-bit error** $E(x) = x^i$: if $G(x)$ has two or more terms it never divides x^i , so **all single-bit errors are detected**.
- **Two isolated errors** $E(x) = x^i + x^j = x^j(x^{i-j} + 1)$: if $G(x)$ is not divisible by x , a sufficient condition to detect all double errors is that **$G(x)$ does not divide $x^k + 1$** for any k up to the maximum frame length. E.g. $x^{15} + x^{14} + 1$ does not divide $x^k + 1$ for any k below 32,768.
- **Odd number of errors**: $E(x)$ then has an odd number of terms, and no polynomial with an odd number of terms has $x + 1$ as a factor modulo 2. Making **$(x + 1)$ a factor of $G(x)$** catches **all** odd-count errors — statistically half of all cases.
- **Burst errors**: a code with r check bits detects **all bursts of length $\leq r$** . A burst of length $r + 1$ is accepted only if it is identical to $G(x)$ — the first and last bits are fixed at 1, so this depends on the $r - 1$ middle bits, giving probability $(\frac{1}{2})^{r-1}$. For bursts longer than $r + 1$, or several shorter bursts, the probability of a bad frame slipping through is $(\frac{1}{2})^r$.

The **IEEE 802 standard CRC-32** (following Ethernet) is $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$. It detects all bursts of length ≤ 32 and all bursts affecting an odd number of bits, and has been used since the 1980s — though not necessarily the best: exhaustive searches by **Castagnoli et al. (1993)** and **Koopman (2002)** found CRCs with **Hamming distance 6** for typical message sizes, whereas IEEE CRC-32 has distance only 4. CRCs are **easy to compute and verify in hardware** with simple shift-register circuits (Peterson & Brown, 1961), with faster implementations appearing regularly (Mitra & Nyack, 2017); hardware is nearly always used, in virtually all LANs (Ethernet, 802.11) and point-to-point links (packets over SONET).

Code	Distance / strength	Mechanism	Detects
Parity (1 bit)	Distance 2	XOR of data bits = one check bit.	Any single-bit error (burst: only 50%)
Interleaved parity	n parity over kn bits	Column parity, data sent row-wise.	Any single burst of length $\leq n$
Internet checksum (16-bit)	Moderate; weak to some patterns	One's-complement sum of 16-bit words, with end-around carry.	Many bit/word errors; misses zero add/delete, splices, swaps
Fletcher's checksum	Stronger than plain sum	Running sum plus a positional (data $\times$ position) term.	Also detects position changes
CRC-32 (IEEE 802)	Hamming distance 4	Modulo-2 polynomial division by $G(x)$.	All bursts ≤ 32 ; all odd-bit errors

3.3 Elementary Data Link Protocols

Three protocols of increasing complexity introduce the subject, each handling a more realistic situation than the last. First, the simplifying assumptions behind the communication model.

3.3.1 Initial Simplifying Assumptions

- **Independent processes**. The physical, data link, and network layers are independent processes that communicate by passing messages. A common split runs the physical-layer process and part of the link-layer process on dedicated hardware — a **NIC (Network Interface Card)** — while the rest of the link layer and the network layer run on the main CPU inside the OS, the link-layer software often taking the form of a **device driver**. Other arrangements exist (all three offloaded to a network accelerator, or all three on the CPU in a software-defined radio), and the preferred one shifts decade

by decade; treating the layers as separate processes keeps the discussion clean and stresses their independence.

- **Unidirectional communication.** Machine A sends a long stream of data to machine B using a reliable, connection-oriented service. A has an infinite supply of data ready and never waits — whenever A's link layer asks, the network layer complies immediately. (Both restrictions are dropped later, when B also sends to A at the same time.)
- **Reliable machines and processes.** Machines do not crash; these protocols deal with communication errors, not with computers crashing and rebooting.

To the link layer, the packet handed down from the network layer is **pure data**, every bit of which must be delivered to the destination's network layer; the fact that the destination may interpret part of it as a header is of no concern to the link layer.

3.3.2 Basic Transmission and Receipt

At the sender, the link layer **encapsulates** a network-layer packet into a **frame** by adding a header and a trailer — so a frame is an embedded packet plus control information (header) plus a checksum (trailer). Library procedures `to_physical_layer` and `from_physical_layer` send and receive frames and compute or check the checksum (usually in hardware, e.g. with the CRC of the previous section), so the protocols below need not deal with it.

The receiver starts with nothing to do, waiting via `wait_for_event(&event)`, which returns only when something happens; the variable `event` then says what. When a frame arrives the receiver computes the checksum: a bad one gives `event = cksum_err`; an undamaged frame gives `event = frame_arrival`, after which the frame is fetched with `from_physical_layer`, its header checked, and only the **packet portion** passed up. (In reality an interrupt, not a tight loop, drives this; we ignore such parallelism and assume the link layer is dedicated to one channel.)

The header never crosses into the network layer

Under no circumstances is any part of a frame header given to the network layer. Keeping the network layer completely ignorant of the link protocol and frame format means those can change — e.g. whenever a new NIC is installed — without touching network-layer software. This rigid, well-defined layer-2/layer-3 interface lets protocols in different layers evolve independently.

The file **protocol.h** collects C definitions shared (via `#include`) by every protocol. Five data structures are defined:

Definition	Meaning
boolean	Enumerated type taking the values true and false.
seq_nr	A small integer that numbers frames so they can be told apart; it runs from 0 up to and including MAX_SEQ, which each protocol defines as needed.
packet	The unit exchanged between the network and link layers (and between network-layer peers); in the model it always holds MAX_PKT = 1024 bytes, though realistically it would be variable length.
frame_kind	Enumerated type: data, ack, nak.
frame	A struct with four fields — kind, seq, ack, info — the first three forming the control header, info carrying the network-layer packet.

The **kind** field says whether the frame carries data (some protocols distinguish pure-control frames from data frames); **seq** and **ack** hold the sequence and acknowledgement numbers; **info** holds a single packet in a data frame and is unused in a control frame. The library procedures (implementation-dependent routines) are:

Procedure	What it does
wait_for_event(&event)	Sits in a tight loop; returns only when something happens, placing the event type in event.
from_network_layer(&p) / to_network_layer(&p)	Accept a packet from, and deliver a packet to, the network layer — the layer-2/3 interface.
from_physical_layer(&r) / to_physical_layer(&s)	Receive a frame from, and hand a frame to, the physical layer — the layer-1/2 interface.
start_timer(k) / stop_timer(k)	Turn the retransmission clock for frame k on and off; a timeout yields event = timeout. Calling start_timer while it runs simply resets it to a full interval.
start_ack_timer() / stop_ack_timer()	Control an auxiliary timer used to generate acknowledgements under certain conditions.
enable_network_layer() / disable_network_layer()	Permit or forbid the network layer from causing network_layer_ready events, so the link layer is not swamped with packets it cannot buffer.
inc(k)	A macro that advances k circularly (MAX_SEQ is followed by 0); written as a macro because it sits on the performance-critical path.

In most protocols the channel is **unreliable** and occasionally loses entire frames, so the sending link layer starts a timer whenever it sends a frame; if no reply arrives within a predetermined interval, the clock times out (**event = timeout**) and the sender is interrupted.

3.3.3 Simplex Link-Layer Protocols

Protocol 1 — Utopia: no flow control or error correction

As simple as possible, because nothing can go wrong: data flow in **one direction only**, both network layers are always ready, processing time is ignored, buffer space is infinite, and the channel **never damages or loses frames**. The sender loops forever fetching a packet and firing out a frame; the receiver loops forever waiting for a **frame_arrival**, fetching the frame, and handing the data up. There are no sequence numbers and no acknowledgements. Thoroughly unrealistic, it exists only to show the basic structure.

Protocol 2 — Stop-and-Wait: adding flow control

The channel is still error-free and traffic still simplex, but now the receiver has **finite buffering and speed**, so the sender must be stopped from flooding it. One option is brute force — build the receiver (or throttle the link) to handle a continuous stream of back-to-back frames at line rate — but that needs dedicated hardware, wastes resources at low utilization, and merely shifts the too-fast-sender problem up to the network layer. The general solution is **receiver feedback**: after passing a packet to its network layer, the receiver returns a tiny **dummy (acknowledgement) frame** that effectively grants permission to send the next frame, and the sender must wait for it before continuing.

Protocols where the sender transmits one frame and then waits for an acknowledgement before proceeding are called **stop-and-wait**. Although the data are simplex (sender → receiver), frames travel **both ways**, so the channel must allow bidirectional transfer — but because flow strictly alternates (sender, receiver, sender, receiver, ...), a **half-duplex** channel suffices. The sender need not even inspect the incoming frame, since it is

always an acknowledgement; **receiver2** differs from the Utopia receiver only in sending that acknowledgement after delivering each packet.

Protocol 3 — PAR / ARQ: adding error control with sequence numbers

Now the channel **makes errors** — frames may be damaged or lost entirely. Damaged frames are caught by the receiver's checksum; if a frame is damaged yet still yields a correct checksum (unlikely), this protocol — like all others — can fail and deliver a bad packet upward.

A naive fix adds only a timer: the sender sends, the receiver acknowledges **only correctly received** frames and discards damaged ones, and the sender times out and resends until the frame arrives intact. This has a **fatal flaw** — a lost acknowledgement causes a duplicate:

1. A's packet 1 is received correctly at B, passed to B's network layer, and B sends an acknowledgement.
2. The acknowledgement is **lost completely** — the channel mangles and loses control frames too, not just data frames.
3. A eventually **times out** and, assuming its data frame was lost, resends the frame containing packet 1.
4. The duplicate arrives intact at B and is unwittingly passed up again — part of the file is duplicated with no error detected, and the protocol fails.

The cure is to let the receiver tell a **first-time frame** from a **retransmission** by putting a **sequence number** in each frame's header. How many bits are needed? The only ambiguity is between a frame **m** and its immediate predecessor or successor — never between the predecessor and successor themselves — because the event that triggers sending frame **m + 1** is the arrival of the acknowledgement for **m**, which already implies **m – 1** and its acknowledgement succeeded. Hence a **1-bit sequence number (0 or 1) is sufficient**: the receiver expects a particular number, accepts and passes up a frame bearing it (then flips the expected number modulo 2), and rejects any frame with the wrong number as a duplicate — while **repeating the last valid acknowledgement** so the sender eventually learns the frame got through.

Protocols in which the sender waits for a **positive acknowledgement** before advancing are called **ARQ (Automatic Repeat reQuest)** or **PAR (Positive Acknowledgement with Retransmission)**. Protocol 3 keeps state across the wait: the sender remembers **next_frame_to_send**, the receiver remembers **frame_expected**, and each has a short initialization phase before its infinite loop. After sending a frame the sender **starts the timer** (reset if already running), its interval chosen to cover frame transit, worst-case receiver processing, and acknowledgement propagation — too short an interval sends needless duplicates that hurt performance but not correctness. The sender then waits for exactly one of three events:

- **A valid acknowledgement** — fetch the next packet into the buffer (overwriting the old one) and advance the sequence number.
- **A damaged acknowledgement, or a timeout** — leave the buffer and sequence number unchanged so a **duplicate** is resent.

In every case the buffer contents (next packet or duplicate) are transmitted. At the receiver, a valid frame's sequence number is checked: a new frame is accepted, passed up, and acknowledged; duplicates and damaged frames are **not** passed up, but still cause the last correctly received frame to be acknowledged — signalling the sender to advance or to retransmit.

Protocol	Channel assumed	Problem solved	Key mechanism
1 — Utopia	Perfect (no loss, no errors)	Nothing — shows basic structure	Send and receive in an endless loop; no seq / ack

Protocol	Channel assumed	Problem solved	Key mechanism
2 — Stop-and-Wait	Error-free, finite receiver	Fast sender overrunning a slow receiver	Receiver returns a dummy ack granting the next send
3 — PAR / ARQ	Lossy and error-prone	Damaged / lost frames and duplicates	1-bit sequence number + timer + retransmission

3.4 Improving Efficiency

The previous protocols sent data in **one direction only** and sent **one frame at a time**. Real links usually need **both directions**, and the link layer is far more efficient if it can send **several frames before an acknowledgement returns**. This section develops both ideas, then gives example protocols that realize them.

3.4.1 Goal: Bidirectional Transmission, Multiple Frames in Flight

Bidirectional transmission: piggybacking

One way to get full duplex is to run **two copies** of an earlier protocol, each on a separate simplex link — a forward channel for data and a reverse channel for acknowledgements — but the reverse channel's capacity is then almost entirely wasted. Better to use **one link for both directions**, intermixing A→B data frames with B→A acknowledgement frames; the **kind field** in each header tells the receiver whether an arriving frame is data or an acknowledgement.

A further improvement: when a data frame arrives, the receiver **restrains itself** instead of immediately sending a control frame, waiting until the network layer hands it the next outgoing packet, and then attaches the acknowledgement to that data frame via the **ack field** — the acknowledgement rides along for free. This temporary delaying of outgoing acknowledgements to hook them onto the next outgoing data frame is **piggybacking**.

Piggybacking: the payoff and the catch

Payoff: better bandwidth use (the ack field costs only a few bits — often just 1 — versus a whole separate frame with its own header, ack, and checksum) and a lighter processing load from fewer frames. Catch: how long should the link layer wait for a packet to piggyback on? Waiting longer than the sender's timeout triggers a needless retransmission. Since the link layer cannot foresee when the next packet will arrive, it uses an ad hoc rule — wait a fixed number of milliseconds; if a packet arrives in time, piggyback; otherwise send a separate acknowledgement.

Sliding windows

The next three protocols are bidirectional **sliding window protocols**, differing in efficiency, complexity, and buffer needs. Each outbound frame carries a **sequence number** from 0 up to a maximum, usually $2^n - 1$ so it fits exactly in an n-bit field (stop-and-wait uses $n = 1$; richer versions use arbitrary n). At any instant the sender keeps a set of sequence numbers it may send — the **sending window** — and the receiver keeps a **receiving window** of frames it may accept. The two windows **need not** share lower/upper limits or even size, and may be fixed or may grow and shrink over time.

These protocols give the link layer freedom in the order it sends and receives, but the requirements are unchanged: packets must still be **delivered to the destination network layer in the order they were handed to the sender's link layer**, and the physical channel is still **wire-like** — it delivers all frames in the order sent. Window mechanics:

- **Sender's window** holds frames sent-or-sendable but **not yet acknowledged**. A new packet from the network layer gets the next sequence number and advances the window's **upper edge**; an incoming acknowledgement advances the **lower edge**. Since windowed frames may be lost in transit, the sender must **buffer all of them** for possible retransmission — a maximum window of n needs n buffers — and if the window fills, the sending link layer must **forcibly shut off the network layer** until a buffer frees.
- **Receiver's window** holds frames it may accept; a frame falling in the window is buffered. When the frame whose number equals the window's **lower edge** arrives, it is passed up and the window **rotates by one**; frames outside the window are discarded. Every case generates an acknowledgement. A window of **1** means the receiver accepts frames only **in order**; larger windows do not — but the network layer is always fed data in the proper order regardless. Unlike the sender's, the receiver's window **stays its initial size**, rotating as frames are accepted.

3.4.2 Examples of Full-Duplex, Sliding Window Protocols

Protocol 4 — One-bit sliding window

A sliding window of size 1, so it is **stop-and-wait**: the sender transmits a frame and waits for its acknowledgement before the next. Variables `next_frame_to_send` and `frame_expected` are each 0 or 1. One side must go first (only one program places the `to_physical_layer` and `start_timer` calls outside the main loop). On each arrival the receiver checks for a duplicate as in Protocol 3; an in-order frame is passed up and the window slides. The **ack field carries the number of the last correctly received frame**; if it matches the frame the sender is trying to send, the sender moves on to the next packet, otherwise it keeps retrying. A frame is sent back whenever one is received.

Protocol 4 is correct — **no combination of lost frames or premature timeouts** can deliver a duplicate to a network layer, skip a packet, or deadlock. But a subtlety: if **both sides start simultaneously**, their first frames cross and half the frames carry duplicates **even with no transmission errors** (in the normal case, where one side waits for the other's frame first, every frame is accepted and there are no duplicates). Premature timeouts can cause the same effect, and repeated premature timeouts can send a frame three or more times, wasting bandwidth.

Motivation for pipelining: the bandwidth-delay product

Stop-and-wait collapses when the round-trip time is not negligible. Consider a **50-kbps satellite link with a 500-ms round-trip delay** sending 1000-bit frames: at $t = 0$ the sender starts, at $t = 20\text{ ms}$ the frame is fully sent, at $t = 270\text{ ms}$ it has fully arrived, and at $t = 520\text{ ms}$ the acknowledgement returns (best case). The sender was blocked $500/520 \approx 96\%$ of the time — only **4% of the bandwidth** was used. Long transit time + high bandwidth + short frames is disastrous.

The fix is to relax the wait-for-ack rule and let the sender transmit up to **w frames** before blocking. Choose w from how many frames fit in the channel: the **bandwidth-delay product** (bandwidth in bits/s $\times$ one-way transit time), divided by frame size, gives **BD frames**; set **$w = 2BD + 1$** (twice the bandwidth-delay is what can be outstanding over a full round trip; the $+1$ is because an acknowledgement is not sent until a whole frame is received). For the example (50 kbps, 250-ms one-way), $BD = 12.5\text{ kbit} = 12.5\text{ frames}$, so **$2BD + 1 = 26$** : sending a frame every 20 ms, by $t = 520\text{ ms}$ (26 frames out) the acknowledgement for frame 0 just arrives, and thereafter acks arrive every 20 ms, keeping 25-26 frames always outstanding.

$$\text{link utilization} \leq w / (1 + 2BD)$$

This upper bound (it ignores processing time and treats the ack as zero-length) shows a **large w is needed whenever BD is large** — high delay, or high bandwidth (a 1-Gbps link with 1-ms delay holds 1 megabit), exhausts a small window quickly. With stop-and-wait ($w = 1$), even one frame's worth of propagation delay drops efficiency below **50%**. Keeping multiple frames in flight is **pipelining**, and it raises a hard question: if a frame mid-stream is lost, many later frames reach the receiver before the sender notices — what should the receiver do with the good frames following the bad one, given that it must deliver to the network layer **in sequence**? Two approaches:

Go-Back-N (Protocol 5)

The receiver uses a **window of size 1**: it discards every frame after the lost/damaged one and sends no acknowledgements for them. The pipeline drains; eventually the sender **times out and retransmits all unacknowledged frames in order**, starting from the lost one. This **wastes bandwidth** when errors are frequent. (In the figure, frames 0 and 1 are acknowledged, frame 2 is lost, the sender keeps sending until frame 2's timer expires, then resends 2, 3, 4, ... from scratch.)

Protocol 5 sets **MAX_SEQ = 7** and, for the first time, **drops the assumption that the network layer always has a packet**: the network layer signals a **network_layer_ready** event, and the link layer uses **enable_network_layer** / **disable_network_layer** to throttle it and enforce the window limit. Key properties:

- **Max outstanding = MAX_SEQ, not MAX_SEQ + 1**, even though there are $\text{MAX_SEQ} + 1$ distinct numbers ($0 \dots \text{MAX_SEQ}$). With $\text{MAX_SEQ} = 7$: if the sender sends 8 frames (0-7), gets a piggybacked ack for 7, sends another 8 (0-7), and another ack for 7 arrives, it **cannot tell** whether the second batch all arrived or all were lost — both leave the receiver acknowledging 7. Hence the cap at MAX_SEQ .
- The sender still **buffers all unacknowledged frames**, since it may have to retransmit them later.
- **Cumulative acknowledgement**: an ack for frame n automatically acknowledges $n - 1, n - 2, \dots$ too — valuable when earlier ack-bearing frames were lost. Whenever an ack frees buffer space, a previously blocked network layer may resume causing **network_layer_ready** events.

Selective Repeat (Protocol 6)

Go-back-n is fine when errors are rare but wastes bandwidth on a poor line. **Selective repeat** instead lets the receiver **accept and buffer good frames received after a bad one** (receiver window > 1). A bad frame is discarded but the good ones following are kept; on timeout the sender retransmits **only the oldest unacknowledged frame**, and once it arrives the receiver delivers all buffered frames to the network layer **in sequence**. This can demand **large link-layer memory**. Selective repeat is usually paired with a **NAK (negative acknowledgement)**, sent on a checksum error or an out-of-sequence frame, which **triggers retransmission before the timer expires**, improving performance. (In the figure: frame 2 is lost; when 3 arrives the receiver NAKs 2 and buffers 3, 4, 5; the NAK brings an immediate resend of 2; the receiver then delivers 2-5 in order and acknowledges up through 5. A lost NAK is harmless — the sender eventually times out on 2.)

Why the window must be at most half the sequence space

With a 3-bit number (0-7) and a window of 7: the sender sends 0-6, all arrive, the receiver advances its window to accept 7, 0, 1, 2, 3, 4, 5 — then a lightning strike wipes out all the acknowledgements. The sender times out and resends frame 0, which now falls inside the receiver's NEW window and is wrongly accepted as new, so the network layer gets a duplicate and the protocol fails. The new range overlapped the old one. The cure: make the maximum window at most **HALF** the sequence range — for Protocol 6, $\text{window} = (\text{MAX_SEQ} + 1)/2$ (four, with 3-bit numbers) — so retransmissions (0-3) and new frames (4-7) can never be confused.

Further Protocol 6 details:

- **Buffers = window size**, not sequence range (four buffers for a 3-bit number); frame i goes in buffer $i \bmod (\text{window size})$. Frames i and $(i + \text{window size})$ never sit in the window together. **Timers = buffers**: one timer per buffer; on timeout, that buffer's contents are retransmitted.
- **Auxiliary ack timer**. Protocol 6 no longer assumes heavy reverse traffic for piggybacking. After an in-sequence data frame arrives, `start_ack_timer` runs an auxiliary timer; if no reverse frame appears before it expires, a **separate acknowledgement** is sent (an **ack timeout** event). There is only one such timer, calling `start_ack_timer` while it runs has no effect (it is not reset — its job is a minimum ack rate), and its timeout **must be appreciably shorter** than the data-frame timeout, so a received frame is acknowledged before its retransmission timer fires.
- **NAK bookkeeping**. The receiver is suspicious when a damaged frame or an out-of-order frame arrives. The variable `no_nak` is true if no NAK has yet been sent for `frame_expected`, preventing duplicate NAKs; a lost NAK is harmless (the sender times out anyway).
- **Tight vs loose timers**. When round-trip time is nearly constant, the sender can set a **tight** timer and NAKs add little. When it is highly variable (e.g. sporadic reverse traffic), a tight timer risks needless retransmissions and a loose one idles after errors — so a **loose** timer is set to be safe, and **NAKs then appreciably speed up** recovery.
- **Which frame timed out?** In Protocol 5 it is always `ack_expected` (the oldest). In Protocol 6 there is no trivial answer — outstanding frames can time out in a scrambled order (e.g. 0, 5, 1, 2, 6 → outstanding list 3405126) — so the code assumes a variable `oldest_frame` is set on timeout to say which frame expired.

Feature	Go-Back-N (Protocol 5)	Selective Repeat (Protocol 6)
Receiver window	1 (in-order only)	> 1 — up to $(\text{MAX_SEQ} + 1)/2$
Frames after an error	Discarded	Buffered until the gap is filled
Retransmission on timeout	All unacknowledged frames, in order	Only the oldest unacknowledged frame
Uses NAK?	No	Yes — speeds recovery before timeout
Scarcer resource it saves	Link-layer buffer memory	Channel bandwidth
Buffers / timers	Sender buffers all unacked	Buffers = timers = window size

3.5 Data Link Protocols in Practice

LANs dominate inside a building (Chap. 4), but most wide-area infrastructure is built from **point-to-point lines**. Three common Internet point-to-point situations are examined: packets over **SONET** optical-fiber links (used e.g. to connect an ISP's routers across locations), **ADSL** links on the telephone local loop at the Internet's edge, and **DOCSIS** links on a cable network's local loop — ADSL and DOCSIS connecting millions of homes and businesses. A single standard, **PPP (Point-to-Point Protocol)** — RFC 1661, elaborated in RFC 1662 — carries packets over all of these (and dial-up modems, leased lines, cable modems, ...), though SONET, ADSL, and DOCSIS each **apply PPP differently**.

3.5.1 Packet over SONET

SONET is the physical-layer protocol most used over the WAN optical-fiber backbone (including the telephone system). It provides a bitstream at a well-defined rate — e.g. **2.4 Gbps for OC-48** — organized as **fixed-size**

byte payloads recurring every 125 µs, whether or not there is user data. A framing mechanism is needed to pick occasional packets out of this continuous bitstream, and **PPP, running on IP routers, provides it**.

PPP improves on the earlier, simpler **SLIP (Serial Line Internet Protocol)**, adding error detection, link configuration, support for multiple protocols, and authentication. It has three main features:

1. A **framing method** that unambiguously marks the end of one frame and the start of the next, and that also handles **error detection**.
2. A **link control protocol — LCP (Link Control Protocol)** — for bringing lines up, testing them, negotiating options, and taking them down gracefully.
3. A way to negotiate **network-layer options** independent of the network-layer protocol: a **separate NCP (Network Control Protocol)** for each supported network layer.

PPP's frame format deliberately resembles **HDLC (High-level Data Link Control)**. The key difference: PPP is **byte-oriented** (uses byte stuffing; all frames are an integral number of bytes), whereas HDLC is **bit-oriented** (bit stuffing; allows frames of e.g. 30.25 bytes). A second difference in practice: HDLC provides reliable transmission (sliding window, acknowledgements, timeouts); PPP can too (RFC 1663, for noisy/wireless environments) but **rarely does** — the Internet nearly always uses an **“unnumbered mode”** giving **connectionless, unacknowledged service**.

PPP field	Size	Value / role
Flag	1 byte	0x7E (01111110) marks frame start/end; one flag suffices between frames, and extra flags fill an idle link.
Address	1 byte	Always 11111111 — all stations accept the frame, avoiding any data-link address assignment.
Control	1 byte	Default 00000011, indicating an unnumbered frame. (Address + Control are constant, so LCP can negotiate to omit them, saving 2 bytes.)
Protocol	1 or 2 bytes	Says what the payload is: codes starting with a 0 bit = network layers (IPv4, IPv6, IPX, AppleTalk); codes starting with a 1 bit = PPP config protocols (LCP, the per-layer NCPs). Default 2 bytes, negotiable to 1.
Payload	Variable	Up to a negotiated maximum; default 1500 bytes. Padding may follow.
Checksum	2 or 4 bytes	A CRC; the 4-byte option is the IEEE 802 CRC-32 of §3.2.2, the 2-byte option another industry-standard CRC.

Byte stuffing. If the 0x7E flag appears inside the payload, it is escaped with **0x7D** followed by the original byte **XORed with 0x20** (which flips bit 5) — so 0x7E becomes **0x7D 0x5E**. Frame boundaries can then be found simply by scanning for 0x7E. Destuffing: on seeing 0x7D, drop it and XOR the next byte with 0x20.

PPP over SONET (RFC 2615) uses the **4-byte checksum** as the primary error detection across the physical, link, and network layers, and recommends **not** compressing the Address, Control, and Protocol fields since SONET already runs fast. One unusual feature: the PPP payload is **scrambled** (XORed with a long pseudorandom sequence) before insertion into the SONET payload — SONET needs **frequent bit transitions** for synchronization, which come naturally from voice but not from data (a user could send a long run of 0s), and scrambling makes such a problem extremely unlikely.

Bringing a PPP link up and down (state machine)

DEAD (no physical connection) → on carrier detected → ESTABLISH, where the peers exchange LCP packets to select options (the initiator proposes; the responder accepts, rejects, or counter-proposes). On agreement → AUTHENTICATE, where the two may check each other's identities. On success →

NETWORK, where NCP packets configure the network layer (for IP, the key step is assigning IP addresses to both ends) → OPEN, the data-transport state in which IP packets ride in PPP frames. When finished → TERMINATE, and once the carrier drops → back to DEAD. A failure in ESTABLISH or AUTHENTICATE returns the link to DEAD.

3.5.2 ADSL (Asymmetric Digital Subscriber Loop)

ADSL connects millions of homes at megabit/s rates over the same telephone **local loop** used for plain telephone service. A **DSL modem** at the home sends bits over the local loop to a **DSLAM (DSL Access Multiplexer)** in the telephone company's local office. In the common stack, a home PC sends IP packets to the DSL modem over a link layer like Ethernet; the modem sends them over the local loop to the DSLAM (or an attached router), where the IP packets are extracted into the ISP network and onward to the Internet.

Reading the ADSL link stack bottom-up: the **ADSL physical layer** uses **OFDM (orthogonal frequency division multiplexing)**, also called discrete multitone (DMT); near the top, just below IP, sits the **same PPP** as for SONET; and **between ADSL and PPP sit two new protocols, ATM and AAL5**.

ATM (Asynchronous Transfer Mode), designed in the early 1990s amid enormous hype as a single system to merge voice, data, cable TV and everything else, never took over (undone, like OSI, by timing, technology, implementation, and politics) but was more successful than OSI and survives in niches — some DSL access lines and especially WAN links inside telephone networks. It is a link layer based on **fixed-length cells**; “asynchronous” means cells are sent **only when there is information to carry** (unlike the continuous SONET bitstream). It is **connection-oriented** — each cell carries a **virtual-circuit identifier** that devices use to forward it along an established path. Cells are **53 bytes = a 48-byte payload + a 5-byte header**; small cells let ATM divide link bandwidth into fine slices (useful for mixing voice and data without long packets causing voice-delay variation). The odd 53-byte length reflects the political design — the 48-byte payload was a compromise between Europe (which wanted 32) and the U.S. (which wanted 64).

Data are mapped into cells by an **ATM adaptation layer** in a process of **segmentation and reassembly**; the main one for packet data is **AAL5 (ATM Adaptation Layer 5)**. An AAL5 frame uses a **trailer** (not a header) giving the length and a **4-byte CRC** — the same CRC as PPP and IEEE 802 Ethernet, shown by Wang & Crowcroft (1992) to be strong enough to detect even cell reordering — plus **padding** that rounds the frame to a multiple of 48 bytes so it divides evenly into cells. No addresses are needed, since each cell's virtual-circuit identifier routes it.

PPPoA (PPP over ATM) (RFC 2364) is not really a protocol but a specification for using PPP with AAL5. Only the PPP **Protocol** and **Payload** fields go into the AAL5 payload; the Protocol field tells the far-end DSLAM whether the payload is an IP packet or something like LCP, and the far end knows the cells carry PPP because an ATM virtual circuit is set up for it. Inside AAL5, **PPP framing is unnecessary** (ATM/AAL5 already frame) and the **PPP CRC is unnecessary** (AAL5 carries the same CRC). This supplements the ADSL physical-layer coding — a **Reed-Solomon code** for correction plus a **1-byte CRC** for any remaining errors — a far more sophisticated error-recovery scheme than over SONET, because **ADSL is much noisier**.

3.5.3 DOCSIS (Data Over Cable Service Interface Specification)

DOCSIS has two components: the **physical (PHY) layer** (sometimes the PMD, physical-media-dependent sublayer) and the **MAC (Media Access Control) layer** (Chap. 4). Above the physical layer DOCSIS handles **bandwidth allocation upstream and downstream (flow control), framing, and error correction**.

A DOCSIS frame carries quality-of-service indicators and supports fragmentation or concatenation. Each **unidirectional** sequence of frames is a **service flow**; primary service flows let the **CMTS (Cable Modem**

Termination System, in the cable company's office) send management messages to each modem. Each flow has a unique identifier and often a **service class** — **best effort**, **polling** (the modem explicitly requests bandwidth), or **grant service** (the modem sends bursts at a guaranteed rate). A **primary (default)** flow carries everything not otherwise classified; many setups have just one default upstream and one default downstream flow carrying all user traffic and management. Historically most data was assumed to flow **downstream**, though video conferencing and cloud gaming (Stadia, GeForce Now, xCloud, targeting 30-35 Mbps streams) push the other way.

On power-up a modem connects to and **registers** with the CMTS, acquiring upstream/downstream channels and encryption keys. The two carriers are **shared channels**: downstream, **every modem receives every packet**; upstream, **many modems transmit to the single CMTS receiver**, and multiple physical paths may exist. Before DOCSIS 3.1, downstream packets were divided into **188-byte MPEG frames** (4-byte header + 184-byte payload — the MPEG transmission convergence layer), with the CMTS periodically sending management info (ranging, channel assignment); 3.1 keeps this only for legacy and no longer relies on it downstream.

Transmission is organized by **modulation profiles** — lists of modulation orders (bit-loadings) for the OFDM subcarriers. Downstream, the CMTS may use different profiles per modem, usually grouping modems of similar performance into one profile. By service-flow ID and QoS, the link layer (the **convergence layer** in 3.1) groups packets of the same profile into the same **send buffer** (one per profile, kept shallow to limit latency); a **codeword builder** then maps each DOCSIS frame to its FEC codewords, pulling from profile buffers only at codeword boundaries. FEC treats the frame as a **bit stream**, and DOCSIS uses an **LDPC codeword** — downstream, a full codeword is up to **2027 bytes (up to 1799 data + 225 parity)**. Within each byte the **least-significant bit goes first**; multi-byte values are sent most-significant byte first (**network order**). For idle fill, the CMTS uses byte stuffing — inserting zero-bit-filled subcarriers into OFDM symbols, or stuffing runs of 1s into codewords.

Since version 3.0, DOCSIS supports **channel bonding** — one subscriber using multiple upstream and downstream channels at once, a form of link aggregation. **DOCSIS 3.0** bonds up to **32 downstream and 8 upstream** channels, each 6-8 MHz wide. **DOCSIS 3.1** uses the same bonding but allows much wider channels (up to **192 MHz downstream, 96 MHz upstream**, vs 6-8 / 6.4 MHz in 3.0) and can bond **across channel types** (e.g. one 192-MHz OFDM channel plus four 6-MHz SC-QAM channels).

	Packet over SONET	ADSL	DOCSIS
Where	WAN optical-fiber backbone	Telephone local loop (home)	Cable local loop (home)
Stack over the link	PPP over SONET	PPP · AAL5 · ATM · ADSL (OFDM/DMT)	DOCSIS link + PHY layers
Framing	PPP flag bytes (0x7E), byte stuffing	ATM/AAL5 cells (no PPP framing)	DOCSIS frames → FEC codewords
Error handling	4-byte CRC-32; payload scrambled	AAL5 CRC + Reed-Solomon + 1-byte CRC (noisier)	LDPC FEC codewords
Key extras	LCP/NCP setup; RFC 2615	PPPoA (RFC 2364); 53-byte ATM cells	CMTS, service flows, channel bonding

3.6 Summary

The data link layer's task is to convert the raw bit stream offered by the physical layer into a stream of **frames** for use by the network layer, and to offer a range of **service qualities** — from connectionless-unacknowledged, through acknowledged, to reliable connection-oriented.

Framing is done by byte-count, byte-stuffing (flag bytes with escapes), or bit-stuffing methods. **Error control** detects or corrects damaged frames and arranges retransmission of lost ones; **flow control** keeps a fast sender from overrunning a slow receiver. The **sliding-window** mechanism integrates both — a window size of one reduces it to the stop-and-wait protocol.

Error-correcting and error-detecting codes add structured redundancy using mathematics. Correcting codes — **convolutional, Reed-Solomon, and LDPC** (in increasing capability) — let the receiver repair errors and are used on noisy channels (and important media). Detecting codes — **CRCs and checksums** — flag errors and trigger retransmission. These codes may be applied at the link layer, the physical layer, and higher layers alike.

A sequence of **ARQ (Automatic Repeat reQuest) protocols** was developed, each adding capability: from an unrealistic error-free channel, to flow control, to error control with sequence numbers and stop-and-wait, to full-duplex **sliding windows with piggybacked acknowledgements**, and finally to **pipelining** — where **Go-Back-N** discards everything after an error while **Selective Repeat** buffers out-of-order frames and uses NAKs for efficiency on unreliable links.

In practice the Internet uses **PPP** as its main data link protocol over point-to-point lines, running in a connectionless-unacknowledged mode: flag bytes delimit frames and a CRC detects errors. PPP carries packets over **SONET** (common in wide-area networks), **ADSL** (connecting homes over the telephone local loop via ATM/AAL5), and over cable via **DOCSIS** — the three practical point-to-point deployments that open-chapter theory ultimately serves.